

- AWK -

- Esempi d'uso -

- AWK: ha bisogno di **due elementi** per funzionare:
 - il flusso in input (file_testo)
 - e il codice per elaborarlo (codice_awk).

Quindi:

```
awk -opzioni '{ codice_awk }' file_testo
```

Oppure

```
cat file_testo | awk -opzioni '{ codice_awk }'
```

Invocazioni

1. `awk programma file-dati`

```
awk '{ print "hello" }' pippo.txt
```

Esegue il programma sul contenuto del file

1. `awk programma`

Esegue il programma su *stdin*

1. `awk -f file-programma file-dati`

2. `awk -f file-programma`

Esegue il programma contenuto nel primo file sul contenuto del secondo file (o su *stdin*)

Sintassi di un Programma

Sequenza di *regole*:

[pattern] [azione]

- Pattern:

- espressione booleana $a \geq 1$
- espressione regolare `/ciao/`

- Azione:

- blocco di istruzioni `{ a = b + 1; print $0 }`

Semantica

Sequenza di *regole*:

[pattern] [azione]

1. Divide l'input in righe
2. Per ogni riga valuta tutte le regole in ordine
3. Esegue tutte le azioni il cui pattern risulta “vero”

Esempio

```
awk '/a/{ print "trovato" }
```

Stampa “trovato” per ogni riga che contiene almeno una “a”

Variabili Speciali

- \$0 riga corrente
- \$1...\$9 colonna i-esima della riga corrente
- NF numero di colonne nella riga corrente (*number of fields*)
- NR numero della riga corrente

Variabili modificabili:

- FS separatore di colonna (*field separator*)
- RS separatore di riga (*row separator*)

Cambiare il Separatore di Colonna

- Default: spazi e tabulazioni
- Cambiare separatore internamente: `FS=" : "`
- Cambiare separatore esternamente:

```
awk -F<separatore> ...
```

- Esempio:

```
awk -F" : " '{ print $1 }' /etc/passwd
```


Esercizio

Senza utilizzare il comando **sed** (solo **awk** e **sort**) a partire dal file dei gruppi stampare il file dei punteggi per gruppo:

Input: NomeGruppo; Punteggio

Iso01InfoNa03;69,00

Iso01InfoNa08;70,00

Iso01InfoNa21;65,33

Iso01InfoNa15;63,56

.....;

Output: Punteggio NomeGruppo

70,00 Iso01InfoNa08

69,00 Iso01InfoNa03

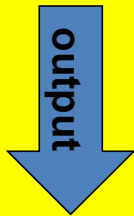
65,33 Iso01InfoNa21

63,56 Iso01InfoNa15

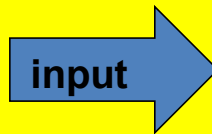
.....

Soluzione Esercizio

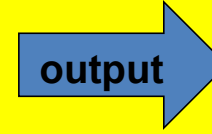
```
awk -F';' '{print $2 " " $1}' gruppiLSO.txt | sort -nr
```



```
69,00 Iso01InfoNa03  
70,00 Iso01InfoNa08  
65,33 Iso01InfoNa21  
63,56 Iso01InfoNa15
```



sort -nr



```
70,00 Iso01InfoNa08  
69,00 Iso01InfoNa03  
65,33 Iso01InfoNa21  
63,56 Iso01InfoNa15
```

Esercizio

Facendo uso del comando **awk**, elencare gli utenti diversi che usano “**bash**” come shell di default presenti nel file /etc/passwd

Esempio:

```
root:x:0:0:root:/home/root:/bin/bash
```

Soluzione

```
cat /etc/passwd | awk 'BEGIN {FS=":"} ($7==" /bin/bash") {print $1}'
```

Esercizio

Facendo uso del comando awk, elencare tutti i pid e gli username dei processi con CPU% maggiore di 0.1. Stampare quindi l'utilizzo totale di CPU da parte di tali processi

Soluzione

```
ps -eo pid,user,pcpu --no-headers |  
awk '$3 > 0.1 { print $1, $2; sum += $3 }  
END { print "Totale CPU%:", sum }'
```

Esercizio

Facendo uso del comando awk, elencare tutti i pid e ppid dei processi dell'attuale user. Stampare quindi il numero totale di processi

Soluzione

```
ps -u $USER -o pid=,ppid= | awk '{print $1,$2;  
tot+=1} } END {print "totale:" tot }'
```

Operatori di confronto

Operatore	Significato	Esempio	Risultato
==	Uguale a	<code>\$1 == "anna"</code>	vero se il primo campo è esattamente "anna"
!=	Diverso da	<code>\$2 != 0</code>	vero se il secondo campo è diverso da 0
<	Minore di	<code>\$2 < 10</code>	vero se il secondo campo è minore di 10
>	Maggiore di	<code>\$2 > 100</code>	vero se il secondo campo è maggiore di 100
<=	Minore o uguale a	<code>\$3 <= 50</code>	vero se il terzo campo è ≤ 50
>=	Maggiore o uguale a	<code>\$3 >= 20</code>	vero se il terzo campo è ≥ 20
~	Match con regex	<code>\$1 ~ /^a/</code>	vero se il primo campo inizia con "a"
!~	Non match con regex	<code>\$1 !~ /mar/</code>	vero se il primo campo non contiene "mar"

Esempio

```
awk '$1 ~ /^a/ { print $2 }'
```

Stampa la seconda colonna delle righe la cui prima colonna comincia per “a”

```
awk 'match($1, /^a/) { print $2 }'
```

Tilde ~

Su tastiera italiana PC: AltGr + ì

Su tastiera Mac con layout italiano: Alt (o Option) + 5

Esempio

```
awk '$1 !~ /^a/ { print $2 }'
```

Stampa la seconda colonna delle righe la cui prima colonna non inizia per “a”

```
awk '!match($1, /^a/) { print $2 }'
```

Tilde ~

Su tastiera italiana PC: AltGr + ì

Su tastiera Mac con layout italiano: Alt (o Option) + 5

Esercizio

Stampa la terza colonna delle righe la cui seconda colonna inizia e finisce con la prima lettera della prima riga

Funzioni utili in awk

```
substr(stringa, inizio, [lunghezza])
```

Estrae una sottostringa da un campo o variabile.

Inizio: posizione da cui partire (1 = primo carattere).

lunghezza (opzionale): quanti caratteri estrarre.

Se lunghezza è omesso → prende fino alla fine della stringa.

Esempi:

```
substr("ciao", 1, 1) → c
```

```
substr("ciao", 2, 2) → ia
```

```
substr("ciao", 3) → ao
```

Funzioni utili in awk

`next`

- Termina l'elaborazione della riga corrente.
- Passa subito alla riga successiva del file di input.

Utile quando:

- Vuoi saltare righe particolari (es. intestazioni).
- Usi la prima riga solo per inizializzare variabili.
- In breve: “ho finito con questa riga, passa oltre”.

```
awk '/^$/ { next } $1 ~ /^a/ { print $2 }' file.txt
```

```
awk '/^#/ { next } $1 ~ /^a/ { print $2 }' file.txt
```

Esercizio

Stampa la terza colonna delle righe la cui seconda colonna inizia e finisce con la prima lettera della prima riga

```
awk 'NR==1 { chiave = substr($1,1,1); next }  
     match($2, "^"chiave".*"chiave"$") { print $3 }' file.txt
```

Abbreviazioni

Sequenza di regole:

[pattern] [azione]

- Azione senza pattern eseguita su tutte le righe
- Pattern senza azione stampa il rigo
- Pattern “/regexp/” abbrev. di “\$0 ~ /regexp/”
- Azione “print” abbrev. di “print \$0”

Esempio

Tutti questi sono equivalenti:

```
awk '{n++} n>10 { print $0 }'
```

```
awk '{n++} n>10 { print }'
```

```
awk '{n++} n>10'
```

```
awk 'NR>10'
```

Stampano tutto, a cominciare dalla riga 11

Passare variabili di shell ad awk

- Supponiamo `a=7` (variabile bash)
- Per ogni rigo, vogliamo stampare il valore di “a” e poi il primo campo del rigo
 - `awk "{ print $a $1 }"` equivale a `awk "{ print 7 }"`
 - `awk '{ print $a $1 }'` equivale a `awk '{ print $0 $1 }'`

Come si fa?

```
awk -v mia_a="$a" '{ print mia_a $1 }'
```

Esercizio

- Lanciare: `ps -eo user,pid,pcpu,comm`

```
USER  PID %CPU COMMAND
root   1  0.0 systemd
anna  1234  1.5 firefox
anna  2345  0.3 bash
mario 3456  5.0 python
```

- Definire una variabile di shell soglia=1.0
- Stampare solo i processi che usano più CPU della soglia mostrando utente, PID, %CPU

Esercizio

- Lanciare: `ps -eo user,pid,pcpu,comm`

```
USER  PID %CPU COMMAND
root   1  0.0 systemd
anna  1234  1.5 firefox
anna  2345  0.3 bash
mario 3456  5.0 python
```

- Definire una variabile di shell `soglia=1.0`
- Stampare solo i processi che usano più CPU della soglia mostrando utente, PID, %CPU

```
ps -eo user,pid,pcpu --no-headers \
| awk -v soglia="$soglia" '$3 > soglia { print $1, $2, $3 }'
```

Pattern Speciali

- **BEGIN**: esegue l'azione prima di leggere il primo rigo
- **END**: esegue l'azione dopo aver letto l'ultimo rigo

Esempio:

```
awk 'BEGIN { print "Inizio" }  
    { print $1 }  
    END { print "Fine" }'
```

Esercizio

Stampare la dimensione totale di tutti i file della directory corrente con estensione *.txt*

Esercizio

Stampare la dimensione totale di tutti i file della directory corrente con estensione *.txt*

```
ls -l | awk '/.txt$/ { tot += $5 }  
            END      { print tot }'
```

Esercizi

1. Stampare solo le righe dispari
2. Stampare solo il numero di righe che contengono almeno una 'x'
3. Stampare il nome del file più grande della directory corrente

Esercizi

- Stampare solo le righe dispari

```
awk 'NR % 2 == 1' file.txt
```

- Stampare solo il numero di righe che contengono almeno una 'x'

```
awk '/x/ { count++ } END { print count }' file.txt
```

- Stampare il nome del file più grande della directory corrente

```
ls -l | awk 'NR>1 && $1 !~ /^d/ { if ($5 > max) { max=$5; file=$9 } } END { print file }'
```

Esercizio

Convertire in HTML l'elenco della directory corrente

Esempio:

Se la directory contiene i seguenti file:

```
> ls  
a.txt b.txt prova.c
```

l'output del programma awk potrebbe essere il seguente:

```
> ls | awk -f convert-to-HTML.awk  
<HTML><BODY>  
<ul>  
<li>a.txt</li>  
<li>b.txt</li>  
<li>prova.c</li>  
</ul>  
</BODY></HTML>
```

Esercizio

Convertire in HTML l'elenco della directory corrente

```
BEGIN {  
    print "<HTML><BODY>"  
    print "<ul>"  
}  
  
{  
    print "<li>" $0 "</li>"  
}  
  
END {  
    print "</ul>"  
    print "</BODY></HTML>"  
}
```

Uso:

ls | awk -f convert-to-HTML.awk

Array

In awk gli array sono associativi: gli indici possono essere stringhe oltre che numeri. La sintassi è simile a quella degli altri linguaggi di scripting.

Dichiarazione e uso

```
n["mario"] = 5
n["anna"]   = 3
print n["mario"]    # stampa 5
```

Iterazione sugli array

```
for (chiave in n) {
    print chiave, n[chiave]
}
```

Uso tipico Conteggio di occorrenze

Somma di valori raggruppati

Tabelle chiave → valore

Esercizio

Mostrare lo spazio usato **per ogni directory di primo livello** sotto la directory corrente.

Suggerimento:

Usa `du -sh *` per avere la dimensione e il nome.

Esempio di input:

```
du -sh *  
4.0K  file1.txt  
8.0K  src  
16K   dati
```

Esercizio

Mostrare lo spazio usato **per ogni directory di primo livello** sotto la directory corrente.

```
du -sh * | awk '{
    size[$2] = $1 # chiave = nome dir/file, valore = spazio usato
}
END {
    for (x in size)
        print x, size[x]
}'
```

Esercizio

Contare quante volte compare ogni **utente** nei processi in esecuzione.

```
ps -eo user --no-headers
```

Esercizio

Contare quante volte compare ogni **utente** nei processi in esecuzione.

```
ps -eo user --no-headers
```

```
ps -eo user --no-headers \  
| awk '{ utenti[$1]++ } END { for (u in utenti) print u,  
utenti[u] }'
```

Esercizio

Calcolare la percentuale totale di CPU usata da ciascun **utente**.

```
ps -eo user,pcpu --no-headers
```

Esercizio

Calcolare la percentuale totale di CPU usata da ciascun **utente**.

```
ps -eo user,pcpu --no-headers
```

```
ps -eo user,pcpu --no-headers | awk '{ cpu[$1] += $2 } END  
{ for (u in cpu) print u, cpu[u] }'
```

Esercizio

Per ogni utente, stampare:

- quante volte compare (numero di processi)
- la somma totale della CPU usata.

```
ps -eo user,pcpu --no-headers
```


Esercizio

Per ogni utente, stampare:

- quante volte compare (numero di processi)
- la somma totale della CPU usata.

```
ps -eo user,pcpu --no-headers
```

```
ps -eo user,pcpu --no-headers \
| awk '{
    count[$1]++          # conta i processi
    cpu[$1] += $2        # somma la CPU
}
END {
    for (u in count)
        printf "%-10s processi:%2d    cpu_totale:%.1f\n", u,
count[u], cpu[u]
}'
```

Funzioni awk

Si definiscono **fuori** da BEGIN / END / regole, con la sintassi:

```
function nome(arg1, arg2, ...) {  
    # corpo  
    return valore  
}
```

- Possono essere usate dentro le azioni, come in altri linguaggi.
- Supportano sia parametri che variabili locali.

Funzioni awk

Esempio: funzione per calcolare la media

```
function media(somma, n) {  
    return somma / n  
}  
  
{  
    tot += $2  
    count++  
}  
  
END {  
    print "Media:", media(tot, count)  
}
```

non è obbligatorio metterle sopra, ma è consigliato perché rende lo script più leggibile

File dati.txt

```
anna 23  
mario 30  
luigi 27
```

Lancio: `awk -f media.awk dati.txt`

Funzioni awk

Esempio: funzione per calcolare la media

```
function media(somma, n) {  
    return somma / n  
}  
  
{  
    tot += $2  
    count++  
}  
  
END {  
    print "Media:", media(tot, count)  
}
```

File dati.txt

```
anna 23  
mario 30  
luigi 27
```

non è obbligatorio metterle sopra, ma è consigliato perché rende lo script più leggibile

Comandi Esterni in awk

In awk si può lanciare un comando esterno usando una **pipe**:

```
comando | getline variabile
```

- Il risultato del comando viene letto e salvato in `variabile`.
- Dopo aver usato il comando, è buona pratica chiudere la pipe:

```
close(comando)
```

- **Esempio**

```
BEGIN {  
    cmd = "date +%Y"  
    cmd | getline anno  
    close(cmd)  
    print "Anno corrente:", anno  
}
```

Comandi Esterni in awk

Esempio:

Scrivere una funzione awk che restituisca il **numero di file** contenuti in una directory, usando comandi esterni.

```
ls -d */ | awk `
function conta_file(dir) {
    cmd = "ls -l " dir " | wc -l"
    cmd | getline n
    close(cmd)
    return n
}
{
    print $0, "contiene", conta_file($0), "file"
}
```